

1. SCOPE AND DECISION

Eight cores (C0..C7) run fixed programs over eight shared memory lines (lines 0..7). Each core has a private cache; two directory banks keep the caches coherent with an MSI protocol with transient states. Each directory bank has a request queue of capacity Q (requests that arrive to a full queue are refused with Nack), and a refused cache retries after a backoff with base B. The decision: choose the directory placement (option P1..P4, Figure 1), Q in {1,2,3,4} and B in {1,2,4,8}.

Line L's home is directory bank D(L mod 2). Figure 1 gives each node's row and column and the placement options; Figure 2 gives every link's latency. Figures 3 and 4 give the cache and directory controllers. Everything below is normative.

2. NETWORK

Messages travel on three virtual networks: RESP (Data, InvAck, Nack), FWD (FwdGetS, FwdGetM, Inv, PutAck) and REQ (GetS, GetM, PutS, PutM). Routing is XY: a message first moves along its row until it reaches the destination's column, then along the column. A message between two entities on the same node does not use a link and is placed in the destination's input queue at the arrivals stage of the next cycle.

Each directed link sends at most one message per cycle (pipelined: a new message may leave every cycle). A message that entered a router (by arrival or by being sent there) in cycle t may leave on its next link in cycle t+1 or later. Among messages waiting at a router for the same outgoing link, the link sends the one on the highest-priority network (RESP before FWD before REQ), then the one that entered that router earliest, then the one with the lowest serial number. A message leaving in cycle d arrives at the next router in cycle d + latency of that link.

Every message receives a serial number when it is created: 1, 2, 3, ... in creation order across the whole system.

3. CYCLE STRUCTURE

Cycles are numbered from 1. In each cycle, in this order:

- (1) Links: every directed link sends at most one waiting message (Section 2).
- (2) Arrivals: messages whose arrival cycle is now are handled in increasing serial order. A message at its destination node goes to the end of the destination entity's input queue for its network, except that a REQ message arriving at a directory whose REQ queue already holds Q messages is refused (recorded for a Nack). Any other message is placed at the router to wait for its next link.
- (3) Controllers, in the order D0, D1, C0, C1, ..., C7.

Messages created in a cycle are sent from the creator's router no earlier than the next cycle.

4. DIRECTORY STEP (each bank, each cycle)

- (a) If its RESP queue is not empty, it removes and handles the first message (Figure 4, Data column).
- (b) If its REQ queue is not empty, it handles the first message per Figure 4; if Figure 4 says stall, the message stays first in the queue and the bank does nothing more with REQ this cycle (later requests wait behind it).
- (c) It sends a Nack (on RESP) to the sender of each request refused in this cycle's arrivals stage, in the order they were refused. The Nack names the refused request type.

A directory bank keeps, per line, a state (I, S, M, S_D), a sharer set, an owner and the memory value (initially 0). A Data message from the directory carries the memory value and an ack count.

5. CACHE AND CORE STEP (each core, each cycle)

(a) If the RESP queue is not empty, remove and handle its first message (Figure 3; Nack per Section 6).

(b) FWD queue: scan from the first message. If a message's line is blocked, skip it. Otherwise handle it per Figure 3: if Figure 3 says stall, leave it in place and mark its line blocked; if not, remove it, handle it, and stop scanning. At most one FWD message is handled per cycle.

(c) If any refused request has its retry cycle at or before now, resend the one with the earliest retry cycle (ties: lower line) on REQ to the line's home; a PutM carries the line's current data.

(d) Core: if the core is busy, waiting for a miss, or finished, nothing happens. Otherwise it takes its next operation:

WAIT n: the core is busy for this cycle and the next n-1 cycles; its next operation is taken n cycles later.

LD L / ST L with L present in the cache: follow the Load or Store column of Figure 3 for L's state. A hit completes now and the next operation is taken next cycle. "stall" means the operation is retried next cycle. A store in S starts an upgrade (the core waits).

LD L / ST L with L not present: if L's set has an empty way, put L in the lowest-numbered empty way in state IS_D (load) or IM_AD (store), send GetS or GetM to L's home, and wait. If the set is full, choose a victim among the set's ways in state S or M: the one with the smallest LRU stamp (ties: lower way index); send PutS (state S, way goes to SI_A) or PutM with its data (state M, way goes to MI_A) to the victim line's home. The operation is retried next cycle. If no way in the set is in S or M, the operation is retried next cycle.

A core with no operation left at step (d) is finished in that cycle.

Caches have 2 sets of 2 ways; line L uses set $(L \div 2) \bmod 2$. A way leaves the cache (becomes empty) when Figure 3 says "way freed". The LRU stamp of a way is the cycle in which it was last used: a hit, the completion of a waiting load or store on it (including an upgrade), or its allocation, whichever is latest.

Miss completion: a waiting load completes when its line reaches S; it returns the line's data. A waiting store (miss or upgrade) completes when its line reaches M; the core then writes the value $1000 \times (\text{core number}) + (\text{number of stores this core has completed, counting this one})$. The core takes its next operation in the following cycle. A store hit writes the same way.

Data handling in IM_AD and SM_AD (note A): Data from home with ack count n adds n to the line's ack counter (which starts at 0 and may have been lowered by earlier InvAcks); if the counter is then 0 the line goes to M and the store completes, otherwise IM_AD goes to IM_A and SM_AD goes to SM_A. InvAck in IM_A or SM_A (note B) lowers the counter by 1; when it reaches 0 the line goes to M and the store completes. A cache sends Data and InvAck messages on RESP and its requests on REQ.

6. NACK AND RETRY

When a cache receives Nack for request type X on line L, it increments that request's refusal count k (k starts at 0 when the request is first sent) and schedules a resend of X on L at cycle $\text{now} + \min(B \times 2^{(k-1)}, 64)$. The line's state does not change. The refusal count is cleared when the request is answered (Data received for the line, or PutAck).

7. WORKLOAD

Each core runs 40 operations generated by: $x(\text{next}) = (1103515245x + 12345) \bmod 2^{31}$, starting from $x = 101 + 7 \times (\text{core number})$. For each operation: advance x once, $r = x \div 256$, kind = $r \bmod 10$; advance x again, $s = x \div 256$; line = $s \bmod 3$ if $((s \div 16) \bmod 10) < 6$, otherwise $s \bmod 8$. kind 0..4: LD line; kind 5..7: ST line; kind 8..9: WAIT $(1 + ((s \div 8) \bmod 7))$.

8. RUN END AND METRICS

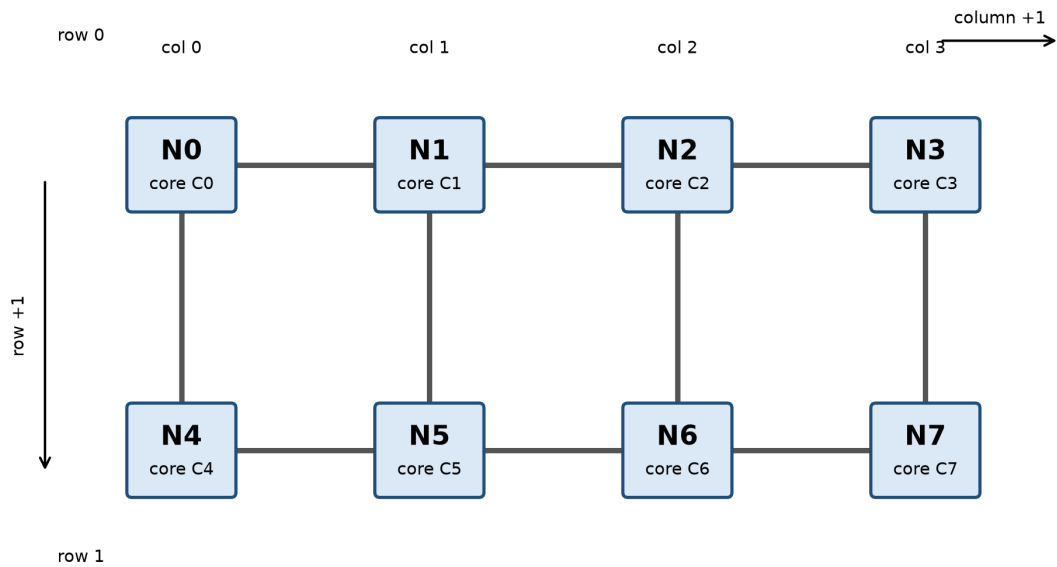
A run ends at the end of the first cycle in which every core is finished, no message is in a queue, router or link, and no resend is scheduled.

Makespan: the latest cycle in which a core became finished. Load-miss latency: completion cycle minus issue cycle, for every load that sent GetS (hits are excluded). p95: sort the latencies ascending and take the element at zero-based index $\min(n-1, \text{floor}(0.95n))$. Messages: the number of messages created (every serial number), all types. Nacks: the number of Nack messages created.

9. SWEEP AND SELECTIONS

Run all 64 configurations (P1..P4 x Q 1..4 x B 1,2,4,8). Fastest: smallest makespan (ties: smaller p95, then fewer messages, then placement name, Q, B). Leanest: among configurations with makespan ≤ 640 , fewest messages (ties: smaller makespan, then placement name, Q, B).

Figure 1. Floorplan: 2 x 4 mesh. Each node has one core with its private cache; a directory bank shares its node's router. Lines are bidirectional links.



Directory placement options (node hosting D0 / node hosting D1): P1: N0 / N7 P2: N1 / N6 P3: N3 / N4 P4: N2 / N5

Figure 2. Link timing: cycles from leaving one router to arriving at the other.

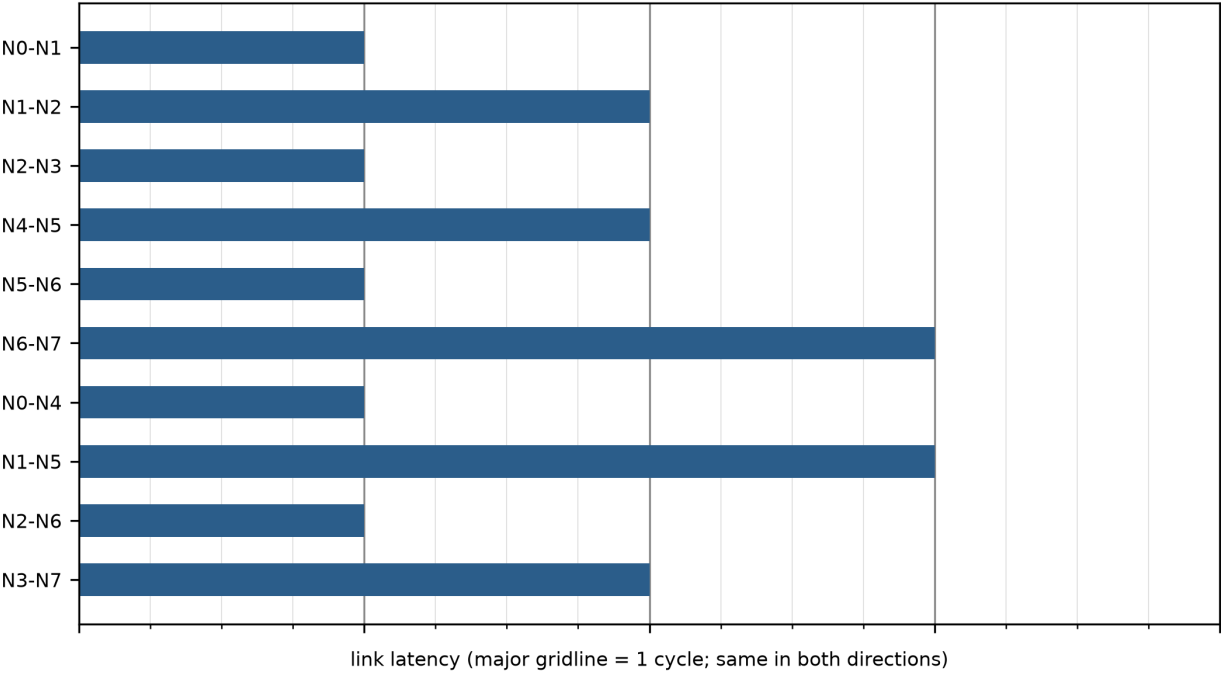


Figure 3. Cache controller (per line). '-' = cannot occur. 'stall' = leave the message (or retry the core operation) as Section 5 says.

state	Load	Store	FwdGetS	FwdGetM	Inv	PutAck	Data from home	Data from a cache	InvAck
I (no way)	miss: GetS to home; IS_D	miss: GetM to home; IM_AD	-	-	-	-	-	-	-
IS_D	stall	stall	-	-	stall	-	-> S; load done	-> S; load done	-
IM_AD	stall	stall	stall	stall	-	-	see note A	-> M; store done	counter -1
IM_A	stall	stall	stall	stall	-	-	-	-	see note B
S	hit	GetM to home; SM_AD	-	-	InvAck to requester; -> I (way freed)	-	-	-	-
SM_AD	hit	stall	stall	stall	InvAck to requester; -> IM_AD	-	see note A	-> M; store done	counter -1
SM_A	hit	stall	stall	stall	-	-	-	-	see note B
M	hit	hit	Data to requester, then Data to home; -> S	Data to requester; -> I (way freed)	-	-	-	-	-
MI_A	stall	stall	Data to requester, then Data to home; -> SI_A	Data to requester; -> II_A	-	-> I (way freed)	-	-	-
SI_A	stall	stall	-	-	InvAck to requester; -> II_A	-> I (way freed)	-	-	-
II_A	stall	stall	-	-	-	-> I (way freed)	-	-	-

Note A: Data from home in IM_AD / SM_AD: add the message's ack count to the line's ack counter; counter 0 -> M and the store completes; otherwise -> IM_A / SM_A.

Note B: InvAck in IM_A / SM_A: counter -1; when it reaches 0 -> M and the store completes. Nack in any state: Section 6. Data from home or a cache sets the line's data.

Figure 4. Directory controller (per line). 'stall' = the request stays first in the REQ queue.

state	GetS	GetM	PutS	PutM	Data (from a cache)
I	Data(ack=0) to req; add req to sharers; -> S	Data(ack=0) to req; owner=req; -> M	PutAck	PutAck	-
S	Data(ack=0) to req; add req to sharers	Data(ack=n) to req, n = sharers other than req; then Inv to each such sharer in ascending core order; clear sharers; owner=req; -> M	remove req from sharers; PutAck; if sharers now empty -> I	remove req from sharers; PutAck; if sharers now empty -> I	-
M	FwdGetS to owner; add req and owner to sharers; clear owner; -> S_D	FwdGetM to owner; owner=req	PutAck	if req is owner: memory=data, clear owner, -> I; PutAck. Otherwise PutAck only	-
S_D	stall	stall	remove req from sharers; PutAck	remove req from sharers; PutAck	memory=data; -> S

Data, FwdGetS, FwdGetM, Inv and PutAck name the original requester where relevant; Inv and Fwd* carry the requester so the cache can answer it. A cache answering FwdGetS sends Data to the requester first, then Data to the home directory.